Mate-Software_workshop_002_guide

Virtual Machines & Docker: Running Linux Tools on Windows

Beginner-friendly guide for Windows and Mac: Docker, an Ubuntu VM, Dockerfile, a first look at ROS2, and Git

Learning outcomes

By the end of this workshop, students should be able to:

- Explain the difference between a VM and a Docker container, and between an image and a container, including what `docker run` does (check, pull, create, start).
- Install Docker Desktop and an Ubuntu VM (WSL on Windows, Multipass on Mac), sign in, and find Images and Containers in Docker Desktop.
- Write a basic Dockerfile, explain each line, build an image and start a container.
- Run the same Python file and a ready-made ROS2 example on their own computer, in the VM and in Docker, and explain why the results differ.
- Push their work to the class GitHub repository and complete three short challenges on their own.

Starting point: this workshop continues from Workshop 001 (VS Code, Git, a GitHub account and uv). It supports Windows and Mac, and purple boxes are for Mac users. Students only need to **copy, run and observe** the ROS2 code. **ROS2 itself will be taught in the next workshop.**

Part 1: Before the workshop

Complete these steps before the workshop. **Mac users:** follow the purple box instead of the Windows-only steps. You need about 15 GB of free disk space and a stable internet connection.

- **Already done in Workshop 001:** VS Code, Git for Windows, GitHub account and uv. Check with `git --version` and `uv --version`.
- **Install Docker Desktop:** https://www.docker.com/products/docker-desktop/ (the Microsoft Store also works). Keep the **WSL 2** option ticked and restart when asked.
- **Install Ubuntu 22.04 in WSL:** open PowerShell **as Administrator** and run the command below. Ubuntu usually starts in the same window and asks for a username and password (Figures 1 and 2). If not, open **Ubuntu 22.04** from the Start menu. Read the box below first.

```
wsl --install -d Ubuntu-22.04
```

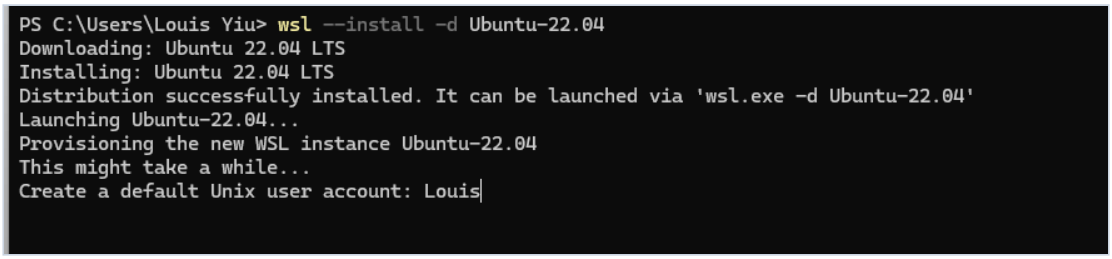


Figure 1: Ubuntu asks you to create a default Unix user account (a username).

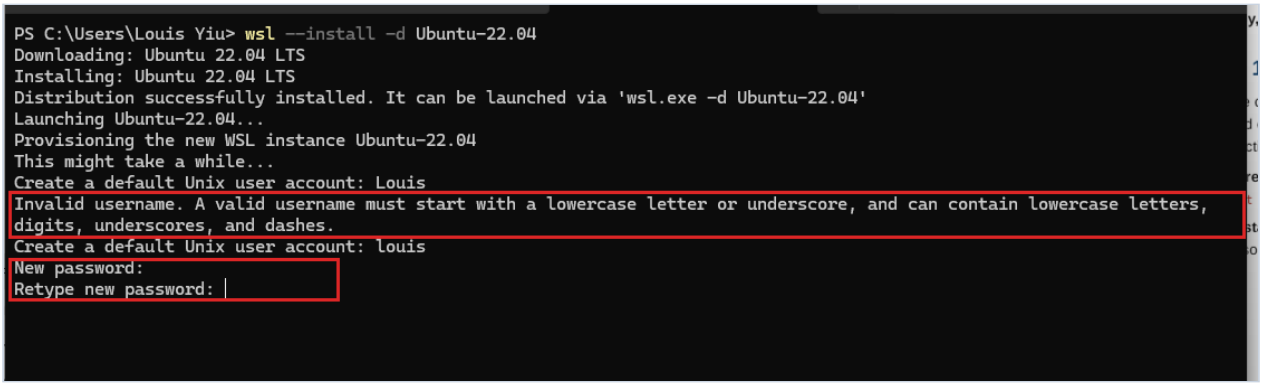


Figure 2: "Louis" was rejected and "louis" accepted. Then Ubuntu asks for the password twice and shows nothing as you type.

Your first time in Ubuntu: please read

- Ubuntu asks for a **username**, then a **password** twice. The username must start with a **lowercase letter** and use only lowercase letters, digits, dashes and underscores (Figure 2).
- **The password is real.** You need it for `sudo` commands in Part 4, so write it down.
- **Nothing appears while you type the password**, not even dots. This is normal. Just type it and press Enter.
- Forgot it? In PowerShell run `wsl -d Ubuntu-22.04 -u root passwd <your-username>`.

Windows only: turn on WSL integration in Docker Desktop (recommended)

This lets the `docker` command work inside Ubuntu too. This guide does not need it, but it is easy to switch on now.

1. In Docker Desktop, click the **gear icon** (Settings), then **WSL integration**. (Older versions: Settings, Resources, WSL integration.)
2. Switch on **Enable integration with my default WSL distro** and **Ubuntu-22.04** (Figure 3), then click **Apply and restart** if it appears.

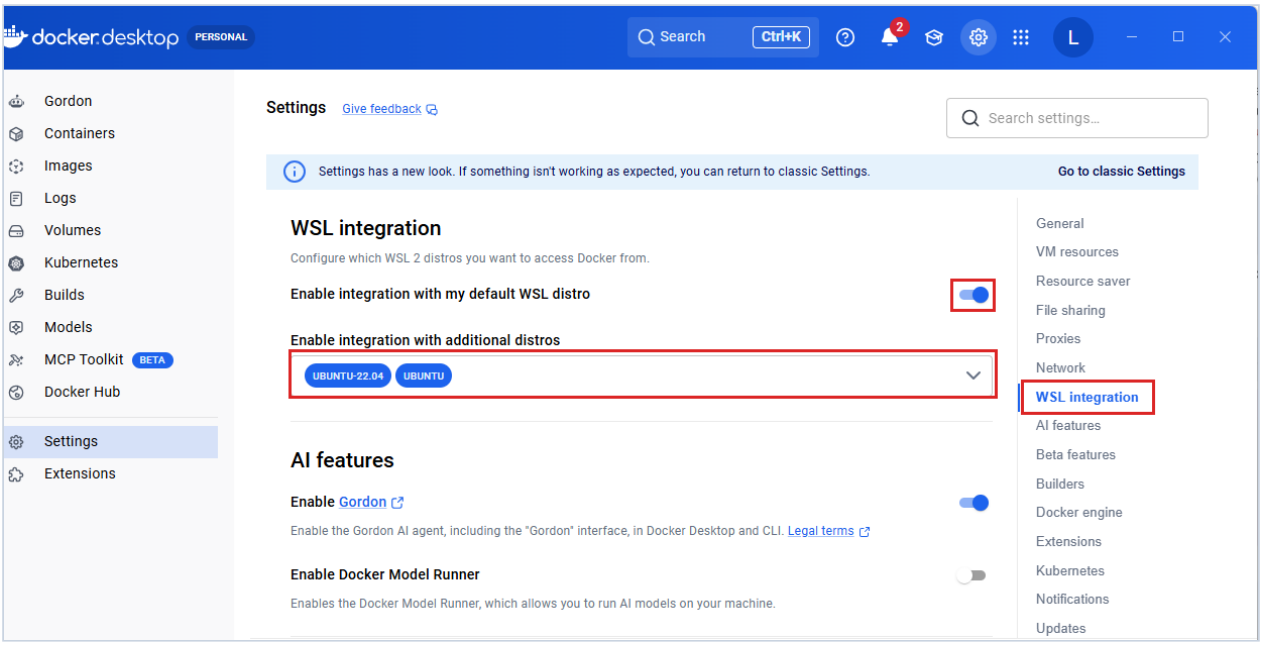


Figure 3: Docker Desktop, Settings, WSL integration. Switch on the default distro and Ubuntu-22.04 (red boxes).

If you cannot turn it on

Some computers cannot enable it, and that is fine. Try in order:

- **Ubuntu-22.04 is not listed:** run `wsl -l -v`. VERSION should be `2`. If it is `1`, run `wsl --set-version Ubuntu-22.04 2`.
- **Switch greyed out, or Docker asks for a WSL update:** run `wsl --update` and reopen Docker Desktop.
- **No WSL integration page:** in Settings, General, tick **Use the WSL 2 based engine** (if shown) and restart Docker Desktop.
- **School or work computers** may block it. Ask the owner, or skip.

You can skip this step. Every Docker command here works in PowerShell without it, and the Ubuntu steps in Part 4 do not need Docker.

Where am I? Look at the start of the line

After the install you may land in the Ubuntu window. **Docker commands in this guide go in the VS Code terminal, not the Ubuntu window.**

The line starts like this	You are in	Use it for
`PS C:\...>`	PowerShell on Windows (also the VS Code terminal)	`docker`, `git`, `uv` commands
`yourname@PCNAME:~$`	Ubuntu (the VM)	Linux commands, `python3`, installing ROS2
`root@abc123:/work#`	Inside a Docker container	The ROS2 steps in Part 4
`yourname@Mac ~ %`	Mac Terminal (also the VS Code terminal)	`docker`, `git`, `uv` commands
`ubuntu@ros-vm:~$`	The Multipass VM on a Mac	Linux commands, `python3`, installing ROS2

Type `exit` to leave Ubuntu, the Multipass VM or a Docker container.
Wrong place? If `docker run hello-world` in Ubuntu gives `permission denied while trying to connect to the Docker API`, type `exit` and run it again in the VS Code terminal.

Other ways to get Ubuntu: the Microsoft Store (search "Ubuntu 22.04"), or https://ubuntu.com/download/desktop with VirtualBox (https://www.virtualbox.org/wiki/Downloads). This guide uses WSL for every "VM" step.

macOS users: do this instead of WSL

- **Docker Desktop for Mac:** same page, https://www.docker.com/products/docker-desktop/ . Choose **Apple Silicon** (M1 to M4) or **Intel chip**. To check: Apple menu, About This Mac, "Chip" or "Processor". Open Docker from Applications.
- **Mac has no WSL.** For our "VM" we use **Multipass**, which creates an Ubuntu VM with one command. Download: https://multipass.run/install (or `brew install --cask multipass`). Another option is UTM: https://mac.getutm.app
- VS Code, Git and uv come from Workshop 001. If missing: VS Code https://code.visualstudio.com/download , Git https://git-scm.com/install , and uv with the command below.

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

Now create the VM (the first launch downloads Ubuntu, so wait a few minutes):

```
multipass launch 22.04 --name ros-vm --cpus 2 --memory 4G --disk 20G
multipass list
```

`multipass list` should show `ros-vm` as **Running**. Where this guide says "WSL Ubuntu", Mac users use `ros-vm`. Apple chips run the ARM version of Ubuntu, and everything works the same.

New to the terminal?

A terminal is a window where you type commands. Type or paste a command, press **Enter**, and wait for the prompt to return. Copy commands exactly, but do not type prompt symbols such as `$` or `PS C:\>`. Press the Up arrow to bring back the last command.

Check before you continue

Open the VS Code terminal and run both commands. `docker --version` should print a version number. `wsl -l -v` should list `Ubuntu-22.04` with VERSION `2`. (Mac: use `multipass list` instead of `wsl -l -v`.)

```
docker --version
wsl -l -v
```

Part 2: Find your way around Docker Desktop

The big idea

A program only works if its surroundings are right: the system, the Python version, the libraries. Docker packs the program **and** its surroundings into one box that works the same on every computer.

Word	What it means	Example
Image	A saved package: a program plus everything it needs. It never changes.	A game installer
Container	A running copy started from an image.	The game, opened
Docker Hub	An online library of ready-made images, such as Python and ROS2.	An app store
VM	A full computer system, such as Ubuntu, running in a window.	A second computer

One image can start many containers. Deleting a container keeps the image.

Sign in

Try this step on your own. Open Docker Desktop, accept the terms, click **Sign in** (top right, Figure 4) and continue with your **Google** or **GitHub** account, or create a Docker account. Your name then shows at the top right.

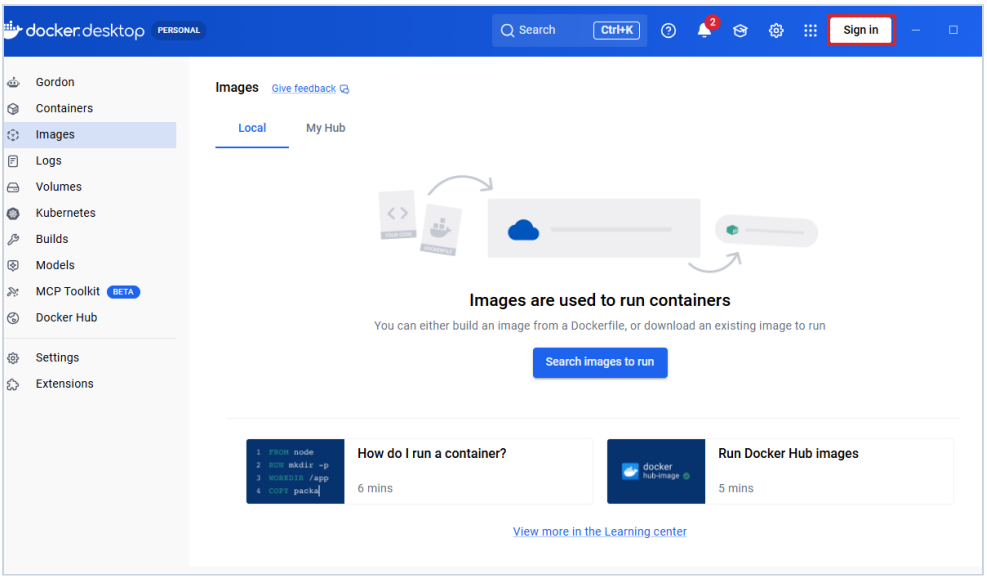


Figure 4: Docker Desktop before signing in. Click the Sign in button (top right, red box).

Run your first container

Open VS Code, open a terminal (Ctrl+`) and run:

```
docker run hello-world
```

You should see "Hello from Docker!" (Figure 5). Behind this one command, **docker run does four things in order**:

1. Check: is image hello-world on my computer? — no, so → 2. Pull it from Docker Hub — then → 3. Create a container from it — then → 4. Start the container

Step 2 shows as the "Pulling from library/hello-world" lines. If the image is already on your computer, Docker skips it, so run the command a second time to see the difference. Each run creates a **new** container from the same image.

```
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
4f55086f7dd0: Pull complete
Digest: sha256:c567b231f65614688711a75f31518a064258a51d442117a3e771a3a8fdba78ef
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
 $ docker run -it ubuntu bash
```

Figure 5: Output of `docker run hello-world` on a computer without the image. Docker's own message lists the steps it took.

Now open Docker Desktop and compare (your version may look slightly different).

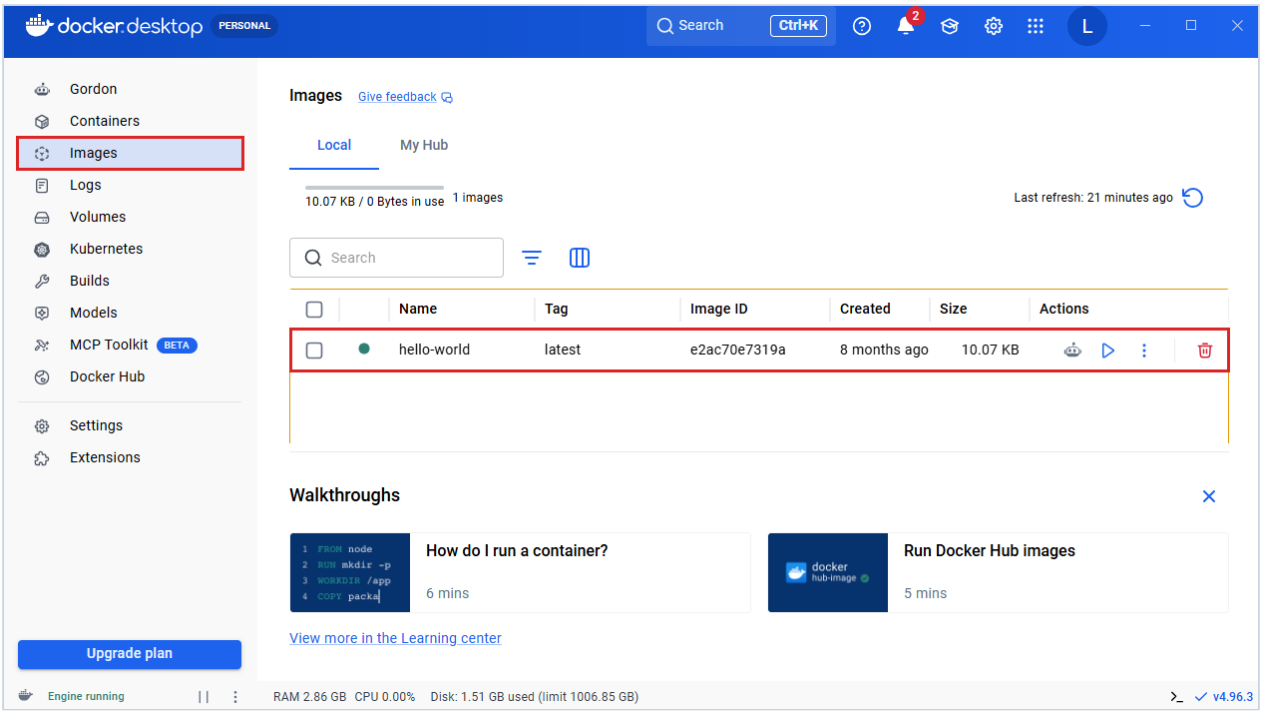


Figure 6: The Images tab lists the images on your computer. "Engine running" (bottom left) means Docker is ready.

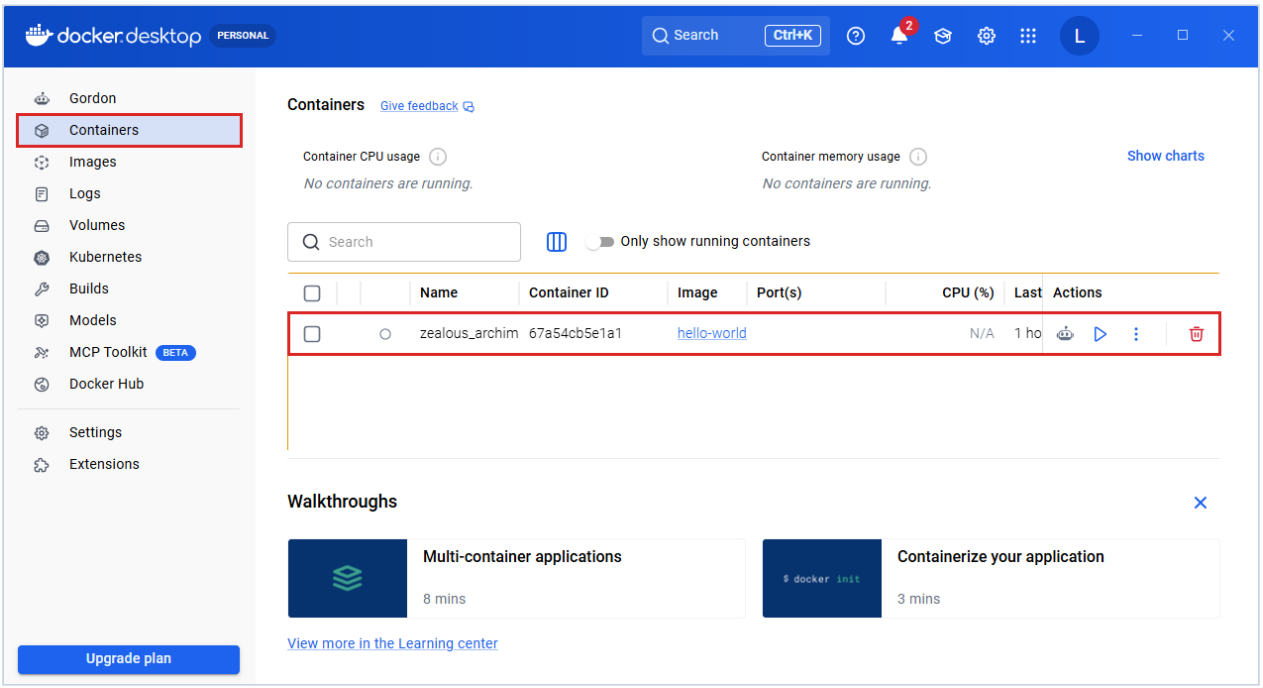


Figure 7: The Containers tab shows the container made from hello-world. It has stopped ("No containers are running") and Docker gave it a random name. (The Volumes tab keeps data after a container is deleted. We do not use it today.)

The terminal can show the same lists:

```
docker images
docker ps -a
```

VM or container?

Both give you a Linux system, in different ways. **A VM is like renting a whole house. A container is like renting one room in a shared house**: smaller and quicker to get, because it shares the house's walls and electricity.

	VM (WSL Ubuntu or Multipass)	Docker container
What is inside	A full system with its own core (kernel)	Just your program and its libraries. It shares the computer's core.
Start time	Slow: seconds to minutes	Fast: usually a few seconds
Size	Big: several GB	Small: often a few hundred MB
Best for	Working inside a whole Linux system	Packing a program so it runs the same everywhere

Part 3: Same file, three places

We run one small Python file on your own computer, in the VM and in Docker, to see that **the same file can behave differently depending on where it runs**.

Open a folder in VS Code

We keep **all workshop files in one folder** called `softwareworkshop`, with a new folder inside it for each workshop. Today's is `Mate-Software_workshop_002`.

```
softwareworkshop
    Mate-Software_workshop_002      (today)
    ...                             (a new folder for each later workshop)
```

Step 1: Create the folders. In File Explorer, go to `C:\`, create a folder named exactly `softwareworkshop`, open it, and create a folder inside named exactly:

```
Mate-Software_workshop_002
```

Use `C:\`, not Documents or OneDrive, so the path is short and both Docker and WSL can find it. You can also move your Workshop 001 folder into `softwareworkshop`.

Step 2: in VS Code, choose File, Open Folder and select `Mate-Software_workshop_002`. **Step 3:** open the terminal with Ctrl+` and check with `pwd` that it starts in this folder.

> **macOS users: folder and terminal**
>
> Create the folders in your home folder (`softwareworkshop` holds every workshop):
>
> ```
> mkdir -p ~/softwareworkshop/Mate-Software_workshop_002
> ```
>
> Open `Mate-Software_workshop_002` in VS Code, then press Ctrl+` for the terminal. It uses zsh, so the commands work as shown unless a purple box says otherwise.

Create the file

Create a new file named `hello.py`, paste this code and save (Ctrl+S):

```
import platform
import sys

print("OS:", platform.system())
print("Detail:", platform.platform())
print("Python:", sys.version.split()[0])
```

Run 1: on your own computer (Windows or Mac)

You have uv from Workshop 001. This file needs no extra packages, so `uv run` works without `uv init`:

```
uv run hello.py
```

Expected output (the details on your computer will differ)

```
OS: Windows
Detail: Windows-11-10.0.26100-SP0
Python: 3.12.x
```

On a Mac the first line says `OS: Darwin` (the name of the macOS core).

Run 2: in the VM (WSL Ubuntu)

In the VS Code terminal, enter Ubuntu and run the file. `/mnt/c` is how Ubuntu sees your C: drive.

```
wsl -d Ubuntu-22.04
cd /mnt/c/softwareworkshop/Mate-Software_workshop_002
python3 hello.py
uname -a
```

Expected output

```
OS: Linux
Detail: Linux-5.15.x-microsoft-standard-WSL2-x86_64-with-glibc2.35
Python: 3.10.x
```

`uname -a` prints Linux system details. It does not exist in PowerShell.

Type `exit` to return to PowerShell.

macOS users: run in the VM (Multipass)

Share your folder with the VM, then enter the VM:

```
multipass mount ~/softwareworkshop/Mate-Software_workshop_002 ros-vm:/home/ubuntu/
work
multipass shell ros-vm
cd ~/work
python3 hello.py
uname -a
```

If the mount is refused, allow `multipassd` in System Settings, Privacy & Security, Full Disk Access, and try again. Output is the same as for WSL. Type `exit` to leave.

Run 3: in Docker

Create a file named exactly `Dockerfile` (no extension) in the same folder:

```
FROM python:3.11-slim
WORKDIR /app
COPY hello.py .
CMD ["python", "hello.py"]
```

A Dockerfile is a **recipe** for making an image. Docker reads one instruction per line, top to bottom.

Line	What the instruction does	In our file
`FROM python:3.11-slim`	Chooses the starting image. Every Dockerfile begins with it.	An official image with Python 3.11 ("slim" means smaller).
`WORKDIR /app`	Sets the folder inside the image where the next lines work, and creates it if needed.	Work in `/app`.
`COPY hello.py .`	Copies a file from your computer into the image (COPY source destination).	Copy `hello.py` into `/app`. The dot means the current folder.
`CMD ["python", "hello.py"]`	The command to run when a container starts.	Run `python hello.py`.



`docker build` turns the Dockerfile into an image, and `docker run` starts a container from it. `FROM`, `WORKDIR` and `COPY` happen when the image is **built**. `CMD` is only stored and runs when a container **starts**. (With `hello-world` we pulled a finished image. Here we build our own.)

Build and run. `-t hello-demo` names the image, and the final `.` means "use the Dockerfile in this folder".

```
docker build -t hello-demo .
docker run --rm hello-demo
```

Expected output

```
OS: Linux
Detail: Linux-5.15.x-microsoft-standard-WSL2-x86_64-with-glibc2.36
Python: 3.11.x
```

The Python version comes from the image, not your computer. `--rm` deletes the container afterwards. `hello-demo` now appears under **Images** in Docker Desktop.

Golden rule to avoid errors

Run commands from the folder that contains your files. If Docker cannot find `hello.py`, check that the terminal is in `softwareworkshop\Mate-Software_workshop_002` and that the file names are exact (not `hello.py.txt`).

Part 4: Run the ROS2 depth example

A small robotics program simulates a depth sensor. One program **publishes** a depth reading every second. A second program **listens** and warns when the depth is above 5 metres.

ROS2 is a robotics framework that normally runs on Linux, so it does not run on plain Windows. **You do not need to understand the code today.** The next workshop teaches it. Copy it, run it, and observe where it works.

Create the two files

Create `depth_publisher.py` in your folder:

```
import random
import rclpy
from rclpy.node import Node
from std_msgs.msg import Float32


class DepthPublisher(Node):
    def __init__(self):
        super().__init__('depth_publisher')
        self.pub = self.create_publisher(Float32, 'depth', 10)
        self.depth = 0.0
        self.timer = self.create_timer(1.0, self.tick)

    def tick(self):
        self.depth += random.uniform(0.0, 0.8)
        msg = Float32()
        msg.data = self.depth
        self.pub.publish(msg)
        self.get_logger().info(f'Published depth: {self.depth:.2f} m')


def main():
    rclpy.init()
    node = DepthPublisher()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        node.destroy_node()
        if rclpy.ok():
            rclpy.shutdown()


if __name__ == '__main__':
    main()
```

Then create `depth_monitor.py`:

```
import rclpy
from rclpy.node import Node
from std_msgs.msg import Float32

LIMIT = 5.0


class DepthMonitor(Node):
    def __init__(self):
        super().__init__('depth_monitor')
        self.sub = self.create_subscription(Float32, 'depth', self.on_depth, 10)

    def on_depth(self, msg):
        if msg.data > LIMIT:
            self.get_logger().warn(f'WARNING: too deep! {msg.data:.2f} m')
        else:
            self.get_logger().info(f'Depth OK: {msg.data:.2f} m')


def main():
    rclpy.init()
    node = DepthMonitor()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        node.destroy_node()
        if rclpy.ok():
            rclpy.shutdown()


if __name__ == '__main__':
    main()
```

Run 1: on Windows or Mac (it should fail)

```
uv run depth_publisher.py
```

Expected output

```
ModuleNotFoundError: No module named 'rclpy'
```

This is the point: Windows and macOS do not have ROS2. Keep this error for your README.

Run 2: in Docker

A ready-made ROS2 image exists, so Docker needs one download. You need **two terminals**: click **+** in the VS Code terminal panel (or press Ctrl+Shift+5).

Terminal 1: start the container (the first run downloads about 1 GB).

```
docker run -it --rm --name ros_demo -v "${PWD}:/work" -w /work ros:humble bash
```

The prompt changes to `root@abc123:/work#`. You are inside a Linux container. Run:

```
source /opt/ros/humble/setup.bash
python3 depth_publisher.py
```

Expected output (Terminal 1), one line per second

```
[INFO] [1730600000.123456789] [depth_publisher]: Published depth: 0.41 m
[INFO] [1730600001.123456789] [depth_publisher]: Published depth: 0.95 m
```

Example output (Terminal 1): depth_publisher.py running in Docker

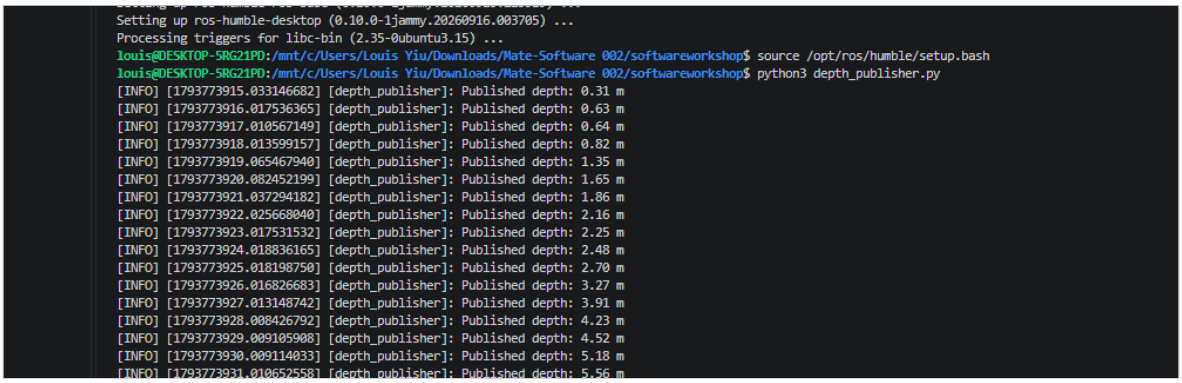


Figure 8: Result of python3 depth_publisher.py after sourcing ROS2 in Docker. The numbers are random.

Continued on the next page: Terminal 2 and Run 3.

Leave Terminal 1 running. **Terminal 2:** enter the same container and start the listener.

```
docker exec -it -w /work ros_demo bash
source /opt/ros/humble/setup.bash
python3 depth_monitor.py
```

Expected output (Terminal 2)

```
[INFO] [1730600002.124] [depth_monitor]: Depth OK: 1.43 m
...
[WARN] [1730600013.126] [depth_monitor]: WARNING: too deep! 5.62 m
```

Warnings start after about 12 seconds, when the depth passes 5 metres. The numbers are random.

Press **Ctrl+C** in both terminals, then type `exit` in Terminal 1. The container is deleted (`--rm`), but your files stay in your folder.

macOS users: one small change in the docker command

Replace `${PWD}` with `$PWD`:

```
docker run -it --rm --name ros_demo -v "$PWD":/work -w /work ros:humble bash
```

Everything else in Run 2 is the same.

Run 3: in the VM (WSL Ubuntu)

Here ROS2 is not pre-installed, so we install it (10 to 20 minutes). Open Ubuntu in a VS Code terminal and run each block in order, entering your Ubuntu password when asked.

```
wsl -d Ubuntu-22.04
```

Step 1: prepare the system

```
sudo apt update
sudo apt install -y software-properties-common curl
sudo add-apt-repository -y universe
```

Step 2: add the ROS2 package source

```
export ROS_APT_SOURCE_VERSION=$(curl -s https://api.github.com/repos/ros-
infrastructure/ros-apt-source/releases/latest | grep -F "tag_name" | awk -F\" '{print
$4}')
curl -L -o /tmp/ros2-apt-source.deb "https://github.com/ros-infrastructure/ros-apt-
source/releases/download/${ROS_APT_SOURCE_VERSION}/ros2-apt-source_$
{ROS_APT_SOURCE_VERSION}.$(. /etc/os-release && echo $VERSION_CODENAME)_all.deb"
sudo dpkg -i /tmp/ros2-apt-source.deb
```

Step 3: install ROS2 Humble

```
sudo apt update
sudo apt install -y ros-humble-ros-base
```

Step 4: run the programs. Open two terminals, enter Ubuntu in each with `wsl -d Ubuntu-22.04`, and in **each** one run:

```
source /opt/ros/humble/setup.bash
cd /mnt/c/softwareworkshop/Mate-Software_workshop_002
```

Then run `python3 depth_publisher.py` in the first and `python3 depth_monitor.py` in the second. The output matches Docker. Stop with Ctrl+C.

> **macOS users: Run 3 in your Multipass VM**
>
> Use `multipass shell ros-vm` instead of `wsl -d Ubuntu-22.04`. Steps 1 to 3 are the same. For Step 4, run this in each terminal, then start the two Python files as above:
>
> ```
> source /opt/ros/humble/setup.bash
> cd ~/work
> ```

Golden rule to avoid errors (ModuleNotFoundError: rclpy)

Every new terminal needs `source /opt/ros/humble/setup.bash` before a ROS2 file. If you see `No module named 'rclpy'` in Docker or WSL, you probably forgot it. If Terminal 2 says `No such container: ros_demo`, Terminal 1 must still be running.

Compare the effort: Docker needed one download and one command, and the VM needed several installation steps. This comparison is the main topic of your README in Part 6.

Part 5: Git and GitHub, push to the class repository

This time we use **Clone Repository**, because the class repository already exists. Your tutor will give you the link and write access. Replace `<repository-url>`, `<repository-name>` and `<your-name>` with the real values.

1. Clone the repository. Open a new PowerShell terminal in VS Code (not Ubuntu or Docker). We clone inside `softwareworkshop`:

```
cd C:\softwareworkshop
git clone <repository-url>
cd <repository-name>
```

If a browser asks you to sign in to GitHub, do so.

2. Create your folder and copy your files into it.

```
mkdir <your-name>
Copy-Item C:\softwareworkshop\Mate-Software_workshop_002\* .\<your-name>\
```

> **macOS users: use these commands for steps 1 and 2**
>
> ```
> cd ~/softwareworkshop
> git clone <repository-url>
> cd <repository-name>
> mkdir <your-name>
> cp -R ~/softwareworkshop/Mate-Software_workshop_002/* ./<your-name>/
> ```
>
> Step 3 (git status, add, commit, push) is the same.

3. Check, save and push.

```
git status
git add .
git commit -m "Add workshop 002 work"
git push origin main
```

If Git asks who you are, run these once and commit again:

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

Refresh the repository page on GitHub to see your folder. If the branch is not `main`, or the push is rejected, ask your tutor.

Part 6: Your own work (Show your understanding here!)

There are no step-by-step instructions here. Use Parts 1 to 5. The **three challenges** get a little harder each time. You may ask GitHub Copilot to explain code, but test everything yourself and explain your results in your own words. Work in your own folder in the cloned repository.

Challenge 1: Compare the environments

Objective: explain results with real evidence.

1. Create `README.md` in your folder.
2. Paste the `hello.py` output from your own computer, the VM and Docker.
3. Make a table comparing **your own computer, the VM and Docker** for the ROS2 program: did it run, the output or error, and how many setup steps it needed.
4. Run `docker images` and note the sizes of `hello-demo` and `ros`. **Guess** which is bigger first, then say if you were right.
5. Answer in 2 to 3 sentences each:
   - Why did the ROS2 program fail on your own computer?
   - What is the difference between an image and a container?
   - Why was Docker quicker to set up than the VM for ROS2?
6. Add a section "What I did not understand yet" about the ROS2 code. This is welcome, and we will cover it next time.

Bonus (optional): time Docker. Windows: `Measure-Command { docker run --rm hello-demo }`. Mac: `time docker run --rm hello-demo`.

Challenge 2: Change the depth monitor

Objective: edit code you did not write and prove it works in two environments.

1. In `depth_monitor.py`, change `LIMIT` to `3.0`.
2. Add a second alert level: above `8.0`, print an **error** that starts with `CRITICAL` instead of the warning. (Hint: the logger has `info`, `warn` and `error`. Only the if/else part changes.)
3. In `depth_publisher.py`, make the robot dive faster, so it reaches 8.0 m within about 30 seconds.
4. Run it in **Docker** and the **VM**. Paste part of each output into your README, showing OK, WARNING and CRITICAL.

> **Example output** (your numbers will differ)
>
> ```
> [INFO] [...] [depth_monitor]: Depth OK: 2.40 m
> [WARN] [...] [depth_monitor]: WARNING: too deep! 4.10 m
> [ERROR] [...] [depth_monitor]: CRITICAL: 8.35 m
> ```

Challenge 3: Build your own ROS2 image

Objective: write a Dockerfile so one command starts the publisher.

1. Create a new file called `Dockerfile.ros` in your folder.
2. Start from `ros:humble`, set a working folder and copy `depth_publisher.py` in. (Hint: compare with the Dockerfile in Part 3.)
3. Set a `CMD` that runs `python3 depth_publisher.py`.
4. Build and run it:

```
docker build -f Dockerfile.ros -t depth-demo .
docker run --rm depth-demo
```

5. In your README, explain in 2 to 3 sentences why this image runs on any computer with Docker, even without ROS2 installed.

> **Example output**
>
> ```
> [INFO] [...] [depth_publisher]: Published depth: 0.37 m
> [INFO] [...] [depth_publisher]: Published depth: 1.02 m
> ```

Commit and push your work

When you finish, push from the cloned repository:

```
git status
git add .
git commit -m "feat: Add environment comparison, depth monitor changes and ROS2
Dockerfile"
git push origin main
```

Wrap-up

By the end, students should have `softwareworkshop\Mate-Software_workshop_002`, a Docker account, `hello.py` and the ROS2 example running in all the environments, and their own folder with README.md and Dockerfiles in the class repository. **Next workshop:** ROS2, including an explanation of today's code.

Prior knowledge: Workshop 001 and a very basic understanding of Python. This session introduces tools and workflow. It does not teach ROS2 or programming from scratch.

Appendix A: How to download ROS2 (Humble)

The ROS2 programs in Part 4 need ROS2 **Humble**. ROS2 runs on Linux, so on Windows or Mac the easiest way is Docker: the ready-made `ros:humble` image already contains it.

Step 1: get ROS2 with Docker

Docker Desktop must be running. In the VS Code terminal (about 1 GB download):

```
docker pull ros:humble
```

Then start a container with the command in Part 4, Run 2 (page 14), and run `source /opt/ros/humble/setup.bash` inside it.

If you see `E: Unable to locate package ros-humble-desktop`

This happens when Ubuntu does not know the ROS2 repository yet, usually because of a missing locale or a skipped repository key step. Run these steps in order inside Ubuntu.

Step 1: set the locale (required for ROS2)

```
sudo apt update && sudo apt install locales -y
sudo locale-gen en_US en_US.UTF-8
sudo update-locale LC_ALL=en_US.UTF-8 LANG=en_US.UTF-8
export LANG=en_US.UTF-8
```

Step 2: add the ROS2 repository and key again

```
sudo apt install software-properties-common -y
sudo add-apt-repository universe -y

sudo apt update && sudo apt install curl -y
sudo mkdir -p /etc/apt/keyrings
sudo curl -sSL https://raw.githubusercontent.com/ros/rosdistro/master/ros.key \
  -o /etc/apt/keyrings/ros-archive-keyring.gpg

echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/ros-archive-keyring.gpg] \
http://packages.ros.org/ros2/ubuntu $(. /etc/os-release && echo $UBUNTU_CODENAME) main" \
  | sudo tee /etc/apt/sources.list.d/ros2.list > /dev/null
```

Step 3: update and install ROS2 Humble

```
sudo apt update
sudo apt install ros-humble-desktop -y
```

Step 4: source and verify

```
source /opt/ros/humble/setup.bash
printenv ROS_DISTRO
ros2 --help
```

`printenv ROS_DISTRO` should print **humble**. Remember: every new terminal needs the `source` line before you run a ROS2 file.